Guion del Livecoding

Chuleta de que decir + comandos + flujo con IA (MCP)

■ Como usar este guion

Las cajas amarillas con ■ son LIBRETO: lo que puedes decir casi literal mientras trabajas. No lo recites de memoria; usalo como ancla para pensar en voz alta. El evaluador valora tu razonamiento tanto como tu codigo.

1. Presentar la plantilla pre-armada

Apenas compartas pantalla, explica que llegas con un framework base ya montado. Esto no es trampa: es como trabaja un QA senior, que no reconstruye su framework en cada tarea.

■ Que decir (libreto)

"Llego con un proyecto base que preparé con la arquitectura que suelo usar: Playwright con TypeScript, patrón Page Object Model, fixtures e integración con Allure. La idea es no gastar tiempo en configuración y enfocarme en automatizar. Déjenme explicarles cómo está estructurado y por qué."

Por que la estructura de la carpeta e2e

Explica que la estructura sigue una separacion por capas (basada en el gTAA de ISTQB): cada carpeta tiene una unica responsabilidad.

Carpeta	Que contiene	Por que separada
e2e/specs/	Los tests (QUE se valida)	Lenguaje de negocio, legible
e2e/pages/	Page Objects (COMO se interactua)	Aisla selectores; si la UI cambia, toco solo aqui
e2e/fixtures/	Inyeccion de los POM	Evita repetir 'new Page()' en cada test
e2e/utils/	Helpers compartidos	Reutilizacion (ej. evidencia)
playwright.config.ts	Entorno y ejecucion	Config, no logica de prueba

■ Que decir (libreto)

"Separo el QUÉ se prueba (specs), del CÓMO se interactúa (page objects), del DÓNDE y con qué se ejecuta (config). Así, si cambia la aplicación toco solo los page objects; si cambia qué validar, solo los specs. Cada motivo de cambio tiene un único lugar: eso es mantenibilidad."

De donde viene y que funcion tiene fixtures/test.ts

Este archivo suele generar dudas, asi que explicalo bien. NO es el `test` original de Playwright: es una version extendida propia.

■ Que decir (libreto)

"El archivo fixtures/test.ts es clave. Importo el test original de Playwright y lo extiendo con mis propias fixtures, que son las que instancian los Page Objects. Por eso en los specs importo el test desde AQUÍ y no desde @playwright/test: así, cuando un test pide 'loginPage', lo recibe ya construido, sin que yo tenga que escribir 'new LoginPage' en cada archivo. Es inyección de dependencias."

```
// fixtures/test.ts  (esencia)
import { test as base, expect } from '@playwright/test';
export const test = base.extend&lt;Pages&gt;({
  loginPage: async ({ page }, use) =&gt; { await use(new LoginPage(page)); },
});
export { expect };

// en el spec:
import { test } from '../fixtures/test';   // &lt;- mi test extendido
test('...', async ({ loginPage }) =&gt; { /* llega listo */ });
```

El helper de evidencia y el config

> **■ Que decir (libreto)**
> *"El helper assertWithEvidence envuelve una aserción en un paso de Allure y, si pasa, adjunta un screenshot. Si falla, corta antes y el screenshot de fallo lo captura el config con screenshot: only-on-failure. En el config también dejé baseURL por variable de entorno y testIdAttribute, que ajusto según la app."*

2. Comandos que usaras (con explicacion)

Generar el proyecto y la carpeta e2e (si tuvieras que crearlo de cero)

```
# Inicializa Playwright + TypeScript y crea la carpeta de tests.
# En el asistente elegí: TypeScript, carpeta de tests 'e2e', GitHub Actions, instalar browsers.
npm init playwright@latest

# Instalar navegadores (si hiciera falta por separado)
npx playwright install chromium
```

■ Que decir (libreto)

"La carpeta e2e la genera el propio asistente de Playwright cuando en 'npm init playwright' le indico ese nombre en vez del 'tests' por defecto."

Explorar y descubrir locators: codegen

```
# Abre un navegador grabador: al interactuar, Playwright SUGIERE el locator que usaria.
npx playwright codegen &lt;URL-de-la-app&gt;

# Ejemplo:
npx playwright codegen https://www.saucedemo.com
```

■ Que decir (libreto)

"Uso codegen para explorar rápido: paso el mouse por los elementos y Playwright me sugiere el locator óptimo. Si me sugiere getByRole o getByLabel, sé que el sitio tiene buena semántica; si cae a CSS, ajusto mi estrategia. Es reconocimiento + borrador a la vez."

Ejecutar los tests: UI, headed y headless

```
npm run test:ui         # modo UI interactivo (recomendado para demostrar): time-travel
npm run test:headed     # navegador VISIBLE, ejecucion normal
npm test                # headless (sin ventana): rapido y como corre en CI
npm run test:smoke      # solo los tests con tag @Smoke
npx playwright test login.spec.ts    # un archivo puntual
```

■ Que decir (libreto)

"Para la demo uso el modo UI porque permite ver cada acción y navegar la línea de tiempo. Pero aclaro que la ejecución real y la de CI corren headless con 'npm test', que es más rápido. El modo visible es para observar, no para producción."

Levantar el reporte Allure

```
# Genera y abre el reporte en un solo paso (ideal en local):
npm run allure:serve

# O generar un HTML estatico y abrirlo:
npm run allure:generate
npm run allure:open
```

■ Que decir (libreto)

"Al terminar corro allure:serve y muestro el reporte: se ven los tests agrupados, cada step con su screenshot, y en los fallos el trace navegable. Recuerdo que Allure necesita Java instalado."

Integrar el MCP de Playwright con Claude

```
# Requiere Node 18+. Registra el servidor MCP oficial de Microsoft en el proyecto:
claude mcp add playwright npx @playwright/mcp@latest

# Iniciar sesion de Claude Code y verificar:
claude
/mcp        # debe listar 'playwright' con sus herramientas
```

■ Que decir (libreto)

"Si me piden integrar IA, registro el MCP oficial de Playwright con Claude Code. El MCP permite que Claude controle un navegador real y vea el árbol de accesibilidad de la página, para generar locators verificados en lugar de inventados."

✓ Chuleta rapida de comandos

Generar: `npm init playwright@latest` | Explorar: `npx playwright codegen <url>` | Ejecutar UI: `npm run test:ui` | Headless: `npm test` | Reporte: `npm run allure:serve` | MCP: `claude mcp add playwright npx @playwright/mcp@latest`

3. Anatomia del playwright.config.ts

Ten claro que hace cada parametro para explicarlo si te preguntan. Se agrupan en tres bloques: opciones de ejecucion, reporteria y ajustes compartidos (use).

Opciones de ejecucion

Parametro	Que hace
testDir: './e2e/specs'	Carpeta donde Playwright BUSCA los tests. Apunto a specs para que no trate los Page Objects como tests.
testIgnore: ['**/_reference/**','**/*.template.ts']	Excluye de la ejecucion las plantillas y el material de referencia.
fullyParallel: true	Ejecuta TODOS los tests en paralelo (no solo los archivos, tambien los tests dentro de cada archivo). Acelera mucho.
forbidOnly: !!process.env.CI	En CI, falla el build si dejaste un test.only olvidado (que saltaria el resto de tests). El !! convierte el valor a booleano.
retries: process.env.CI ? 2 : 0	Reintentos: 2 en CI (mitiga fallos transitorios de red), 0 en local para ver los fallos reales sin enmascararlos.
workers: process.env.CI ? 1 : undefined	Procesos paralelos. 1 en CI (mas estable/reproducible); undefined en local = Playwright decide segun tus nucleos de CPU.

■ Que decir (libreto)

"testDir apunta solo a specs para que los page objects no se confundan con tests. fullyParallel corre todo en paralelo por velocidad. retries y workers los condiciono al entorno: en CI reintento y uso un worker para estabilidad; en local, cero reintentos para ver los fallos tal cual y todos los workers que dé mi máquina."

Reporteria

Parametro	Que hace
reporter: [['line'], ['html'], ['allure-playwright', {...}]]	Array de reportes simultaneos: 'line' en consola, 'html' nativo y Allure. Cada uno recibe sus opciones.
allure resultsDir / detail / environmentInfo	Donde escribe resultados, si muestra hooks, y datos del entorno (SO, Node) en la portada del reporte.

use: ajustes compartidos por todos los tests

Parametro	Que hace
baseURL: process.env.BASE_URL ?? '...'	URL raiz. Permite navegar con rutas relativas (goto('/')). El ?? usa la URL por defecto si no hay variable de entorno.
testIdAttribute: 'data-test'	Atributo que getByTestId debe buscar. Se ajusta o se ignora segun la app.
trace: 'retain-on-failure'	Graba la traza navegable (time-travel) solo cuando un test falla.
screenshot: 'only-on-failure'	Captura la pantalla automaticamente en el momento del fallo.
video: 'retain-on-failure'	Graba video solo si el test falla.
actionTimeout: 10_000	Techo de espera (10s) para UNA accion (click, fill). Falla rapido y claro si un elemento no se vuelve accionable. El _ solo separa miles.

■ Que decir (libreto)

"En use van los ajustes que comparten todos los tests. baseURL me deja usar rutas relativas. trace, screenshot y video solo se activan en fallos, así no inflo el almacenamiento. Y actionTimeout es un techo de 10 segundos por acción para que falle rápido y con un error claro, no para enmascarar lentitud."

projects

Cada 'project' es una configuracion de navegador. Dejo chromium activo y firefox/webkit comentados para activarlos si me piden cross-browser. El `...devices['Desktop Chrome']` es el operador spread: copia toda la config de ese dispositivo.

> **✓ Los cuatro timeouts (no los confundas)**
> **actionTimeout**: una accion (click/fill). **navigationTimeout**: una navegacion (goto). **timeout** (raiz): el test completo (default 30s). **expect timeout**: una asercion web-first (default 5s).

4. Flujo con IA: generar un escenario con Claude + MCP

Como cierre potente, genera un ultimo escenario asistido por IA. El objetivo es demostrar que sabes integrar la IA CON CRITERIO, no que dependes de ella. Sigue estos 4 pasos.

Paso 1 - Dar contexto (el paso que no se salta)

Antes de pedir nada, haz que Claude lea tu arquitectura para que respete tu patron.

```
Lee estos archivos para entender la arquitectura antes de generar nada:
- e2e/fixtures/test.ts
- e2e/pages/SamplePage.template.ts  (o un POM real ya creado)
- e2e/utils/evidence.ts
- e2e/specs/  (algun spec de ejemplo)
```

Paso 2 - El prompt estandar de generacion

✦ Prompt estandar (copiar y adaptar)

Usa **Playwright MCP** para abrir <URL> y explorar el flujo de <describir el flujo, ej. login + agregar item + checkout>. Identifica los locators reales via el arbol de accesibilidad.

Luego genera, siguiendo EXACTAMENTE el estilo de mis archivos existentes:
1. Un Page Object en e2e/pages/<Nombre>Page.ts con locators privados readonly, priorizando getByRole/getByLabel y usando id/name estable solo si hace falta; con las aserciones (expectXxx) DENTRO del POM.
2. El registro de la fixture en e2e/fixtures/test.ts.
3. Un spec en e2e/specs/<nombre>.spec.ts con patron Arrange-Act-Assert, tags @Smoke @<Modulo>, y el helper assertWithEvidence en las validaciones.

Reglas: NO uses waitForTimeout. NO uses XPath posicional. Locators semanticos. Muestrame los cambios ANTES de escribir los archivos.

Paso 3 - Revisar el diff contra tus estandares

Cuando Claude proponga los archivos, revisa ANTES de aprobar:

Revisa que...	Por que
Locators sean semanticos o anclas estables	Evitar fragilidad (nada de XPath posicional ni class con hash)
Aserciones vivan en el POM	Coherencia con tu patron
No haya waitForTimeout	Anti-flaky
El POM este registrado en la fixture	Si no, el spec no lo recibe
No queden imports muertos	Limpieza (ej. expect sin usar)

■ Que decir (libreto)

"Reviso lo que la IA propuso antes de integrarlo: confirmo que los locators son estables, que las aserciones están en el page object y que no metió esperas fijas. Solo entonces apruebo la escritura."

Paso 4 - Ejecutar y validar

```
npm run test:headed    # ver el flujo generado
npm run allure:serve   # revisar evidencia
```

5. Por que la IA es asistencia y no un elemento totalitario

Esta es la parte que mas madurez demuestra. Tenlo claro para explicarlo con seguridad si te preguntan (o dilo proactivamente al usar la IA).

El principio

■ Que decir (libreto)
"Para mí la IA es un copiloto de autoría, no un ejecutor ni el dueño de la arquitectura. Acelera el borrador y la exploración, pero la calidad, el diseño y la decisión final las mantengo yo. Un test generado por IA es un punto de partida, no el entregable."

Cuando SI conviene usarla

Caso de uso	Por que aporta
Explorar una app desconocida	Recorre flujos y revela elementos/estados rapido
Generar un primer borrador de POM/spec	Con MCP produce locators verificados contra el DOM real
Depurar un test flaky	Reproduce el flujo y observa donde se rompe
Descubrir el locator de un elemento dificil	Lee el arbol de accesibilidad por ti

Cuando NO se debe usar (o con cautela)

Limite / caso	Por que
Ejecutar la suite en CI/CD	El pipeline usa el runner (npm test), no un agente: determinismo, velocidad y costo
Como fuente de verdad	Genera borradores; puede equivocarse en locators dinamicos o logica
Cuando ya dominas la app	Escribir a mano es igual de rapido y mas controlado
Volumen alto de tests	El MCP consume mucho contexto/tokens; poco eficiente
Dependencia de hardware (ej. BLE)	La IA no resuelve device fisico; eso se aborda con datasets por API

■ Las limitantes reales de la IA (dilo con honestidad)
Puede alucinar locators o pasos que no existen. Puede introducir malas practicas (waitForTimeout, XPath fragil) si no la corriges. No conoce el contexto de negocio ni el riesgo: no sabe QUE es critico probar. Y no se hace responsable del resultado: esa responsabilidad es tuya.

■ Que decir (libreto)
"La IA tiene límites claros: puede alucinar, no conoce el riesgo de negocio y no asume responsabilidad. Por eso la uso donde acelera sin comprometer calidad —explorar, bosquejar— y la mantengo fuera de la ejecución en CI y de las decisiones de arquitectura. Yo reviso, yo apruebo, yo respondo por el resultado."

Cierre. Si puedes explicar POR QUÉ cada pieza está donde está, por qué elegiste cada locator y por qué la IA es copiloto y no piloto, ya demostraste criterio de QA senior. El código es la mitad; tu razonamiento es la otra mitad. Respira y disfrútalo.